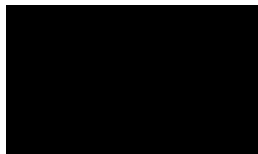


A dissertation submitted to The University of Manchester for the degree of
Master of Science in Advanced Computer Science
in the Faculty of Science and Engineering

Year of submission

2025



Department of Computer Science

Contents

Contents	2
Abstract	4
Declaration of originality	5
Copyright statement	6
Acknowledgments	7
1 Introduction	8
1.1 Background and Motivation	8
1.2 Game Rules	8
1.3 Terminology primer and mathematical background	10
1.4 Literature Review	11
1.5 Objectives and Contributions	13
1.5.1 Objectives	13
1.5.2 Contributions	13
2 Methodology	14
2.1 Design and Technology Selection	14
2.2 System Architecture	15
2.3 Real Time Synchronisation and Multiplayer	16
2.4 AI Agent Designs	17
2.5 Cross-platform testing and validation	19
3 Evaluation and Reflection	20
3.1 Experimental Design	20
3.2 Data Pipeline and Validation	20
3.3 Metric Definitions and Rationale	21
3.4 Results and Visualisation	21
3.5 Per-Grid Narrative	24
3.6 Statistical Perspective	24
3.7 Turn Economy and Tempo	25
3.8 Performance and Product Context	27
3.9 Product UI Overview	27
3.10 User Experience Observations	27
3.11 Limitations	29

3.12 Lessons and Implications	29
4 Project Accomplishment	29
4.1 Overview and Scope	29
4.2 What We Actually Built	30
4.3 Technical Highlights (why this worked)	30
4.4 Research and Product Contributions	30
4.5 Execution Quality	31
4.6 Reflection and Implications	31
4.7 Limitations	31
5 Conclusions and Future Work	32
5.1 Summary of Results	32
5.1.1 Implementation overview	32
5.1.2 Agents and evaluation pipeline	32
5.1.3 Agents and evaluation pipeline	32
5.1.4 Key findings from results	33
5.1.5 Unsafe moves, streak analyses, and performance	33
5.2 Contributions and Implications	33
5.2.1 Research demonstration	33
5.2.2 Architecture perspective	34
5.2.3 Practical implication	34
5.3 Future Directions	34
5.4 Final summary	35
References	35
Appendices	37
A Toolchain & Engine API Surface	37
B Code and Repository	38
C RTDB Paths & Example Snapshot	38
D Benchmark Data Schema (for Reproducibility)	39
E Glossary & User Feedback (Forms)	40

Word count: 8598

Abstract

This project provides an implementation of *Dots and Boxes* that is cross-platform and uses chain-aware AI heuristics to guide play. The module-oriented Dart game engine coupled with Flutter UIs allows us to easily deploy to Android, Web, and Desktop. Three separate AI agents were built: a random baseline, a rule-based agent that avoided immediate traps, and a chain-aware heuristic agent that checked for sequences of connected boxes, utilized the long-chain rule, avoided double-cross scenarios through an evaluation metric, and incorporated two-ply lookahead to plan sacrifice moves. This approach used theory of experts' play; not general search strategies such as minimax or MCTS.

Evaluation used 4×4, 5×5, and 6×6 boards. For each grid we ran 300 games per ordered pairing (A starts vs B, and B starts vs A), plus self-play. For every game we logged who started, the final scores, unsafe-move counts, total turns, and the longest scoring streak. We analysed these logs in Python to produce the heatmaps and plots. In these runs, the chain-aware agent (Deep2) beat the random baseline in all games and also beat the rule-based agent (Heuristic1) in both start orders. Heuristic1 beat Random with a modest edge that depends on who starts. Deep2 took slightly more unsafe moves than Heuristic1 but achieved larger score margins and longer one-turn runs. We did not evaluate deeper search or learning methods here, so our claims are limited to these three agents.

We were able to provide an extensible environment to allow for additional AI strategies to be added, by cleanly separating the logic layer and interface layer within one codebase. This work demonstrates the benefits of including experts in gameplay within the heuristic design, and provides a template for strategic game research.

Declaration of originality

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

I made limited use of generative AI tools - specifically OpenAI ChatGPT and Perplexity, to clarify background concepts and to suggest edits for clarity, grammar, and style. All ideas, code, experiments, figures, and conclusions in this thesis are my own. Any AI suggested wording was reviewed, adapted, and verified by me; no AI-generated text or code has been included. Responsibility for the accuracy and integrity of the work rests entirely with me.

Copyright statement

- i The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made *only* in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the thesis, for example graphs and tables (“Reproductions”), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/>) and in The University’s policy on Presentation of Theses.

Acknowledgments

I thank my supervisor, [REDACTED] for clear guidance and steady feedback. I am grateful to my friends for testing builds and encouraging me throughout. Thanks as well to the open-source communities behind Flutter, Dart, and Firebase, and to the Dots-and-Boxes literature that informed this work. Any errors are mine alone.

1 Introduction

1.1 Background and Motivation

Dots and Boxes, an old pencil and paper game, has long been enjoyed for its strategic depth despite its simple presentation. The game is played on a grid of dots; on each turn a player draws one unused horizontal or vertical edge between adjacent dots. Completing the fourth side of a unit square (“box”) scores a point and grants an immediate extra move [1], [2]. While it is popular as a casual game, both the AI community and combinatorial game theory take it seriously because, beneath the simple rules, the decision space is structured and nontrivial [1].

This project set out to build a fully digital, cross platform Dots and Boxes with interpretable AI agents rooted in expert play. Instead of leaning on heavy search, the agents encode domain heuristics such as chain detection, long chain control, and planned sacrifices so decisions are transparent and fast [1].

To make the system widely available and extensible, we use Flutter for the interface and a pure Dart rules engine. A single codebase targets Android, Web, and Desktop without platform specific forks [3], [4]. For online play, we synchronise game state with Firebase Realtime Database; local multiplayer and AI matches are also supported [5]. Crucially, the rules engine is decoupled from the UI, which lets us run large, deterministic AI versus AI simulations without UI overhead.

The goal is a playable app and a small, inspectable testbed for chain-aware, interpretable AI in a constrained game.

1.2 Game Rules

Players alternate drawing an unused edge between adjacent dots. When a player completes the fourth side of a 1×1 box, they score one point, mark the box as theirs, and immediately take another turn [1], [2] (Fig. 1). The game ends when all edges are drawn; the player with more boxes wins.

Three ideas drive this strategy. First, *chains* are maximal paths of adjacent unclaimed boxes such that, before the chain is opened, each internal box has exactly two open edges and each endpoint has one. A *long chain* has length at least three [6]. A *loop* is a cycle of unclaimed boxes (Fig. 2). Managing when to open and how to harvest these components is fundamental [1], [2]. Second, the *double cross*, also called the “hard hearted handout”, is a planned sacrifice: concede typically two boxes from a long chain to keep control and claim the rest of the chain and often the next component as well [1] (Fig. 3).

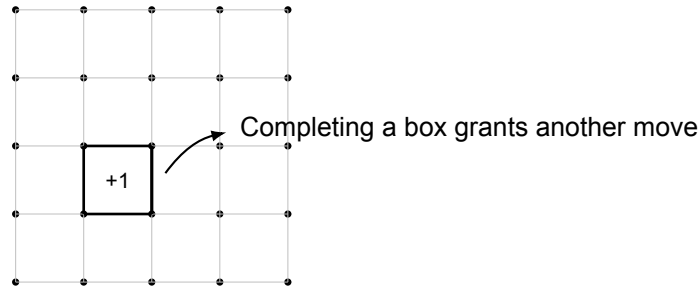


Fig. 1. Completing the fourth edge of a unit box scores one point and grants an immediate extra move [1], [2].

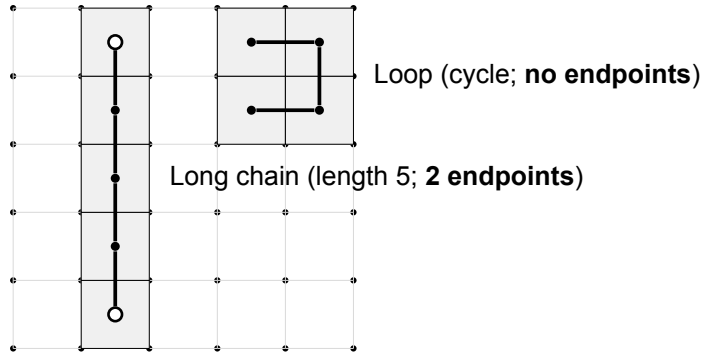


Fig. 2. Endgame components shown as graphs over the box grid. Left: a chain is a path with *two endpoints* (hollow nodes). Right: a loop is a cycle with *no endpoints*.

Third, *parity* matters. The parity of long chains and the presence of loops guide who will be forced into the first unfavourable move. This is often explained via *Nimstring*, a normal play abstraction that connects components to Sprague–Grundy style reasoning. Even though Dots and Boxes is a scoring game, the Nimstring viewpoint explains why giving away two boxes in a long chain can be optimal to preserve control [1], [2]. Together, these ideas create a setting where optimal play balances immediate gains against long term control, which makes the game a strong domain for AI that combines tactical evaluation with strategic planning.

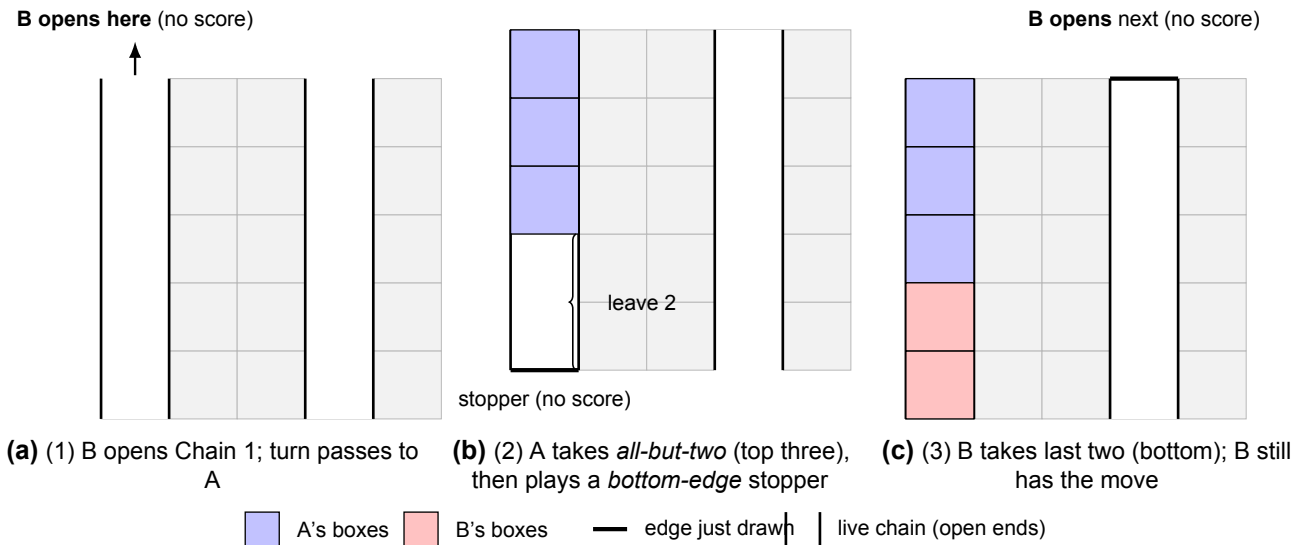


Fig. 3. Double-cross with ends explicitly open. (1) **B** opens at the top of Chain 1 (no score). (2) **A** captures the *top three* (all-but-two) and plays a *bottom-edge* stopper (still no score). (3) **B** is forced to take the remaining *bottom two* and still has the move, so must open the next component.

1.3 Terminology primer and mathematical background

Nimstring. Following [1], Nimstring is a normal play abstraction of Dots and Boxes that replaces boxes by coins and edges by strings. A move cuts a string. When a coin becomes isolated the player who cut the last incident string removes the coin and moves again. The point of the abstraction is that endgame components such as chains and loops behave like impartial games in the Nim sense. Values can be analysed component by component, and the idea of keeping or giving up control becomes a statement about how those component values interact.

Sprague Grundy theory and nimbers. For any finite impartial game under normal play, Sprague Grundy theory assigns a nonnegative integer called the nimber to each position. The nimber of a position is the mex of the nimbers of all options, and disjoint sums of positions combine by bitwise xor of nimbers. In the Nimstring view, chains and loops each have a nim like contribution, so the xor of component values indicates whether the side to move is in a taking position or a leaving position [1]. Dots and Boxes is a scoring game, not normal play, so the mapping is a conceptual bridge rather than an exact identity. It still explains why giving two boxes in a long chain can be correct in order to keep control.

Chains, long chains, and loops. A chain is a maximal path of adjacent unclaimed boxes such that before it is opened each internal box has two open edges and each endpoint has one. A long chain has length at least three. A loop is a cycle of unclaimed boxes. Managing when to open and how to harvest these components is central to expert play [1], [2]. See Fig. 2.

Control, opener, and the hard hearted handout. The opener is the player who is forced to make the first move in a component that gives boxes to the opponent. The controller is the player who plans to keep moving after the opponent's forced captures. In a long chain the controller usually takes all but two boxes and then deliberately hands over the last two. This hard hearted handout maintains control so that the opponent, having just scored, must open the next component. If there is no next component, the controller simply takes all boxes. See Fig. ?? and [1].

Parity and component order. Parity means counting features that decide who must make the first unfavourable opening. Long chain parity and the presence or absence of loops determine whether a handout is needed to keep control. Intuitively, if you would be the player to open the next component after taking everything now, you should pay two boxes in the current chain to flip that obligation to your opponent [1], [2].

Controlled value and terminal bonus. Allcock formalises endgames that are made of chains and loops using two summary quantities. The controlled value is a predicted margin for the controller assuming they keep control to the end. The terminal bonus adjusts for what happens in the last component, for example the fact that the last chain is taken in full. These quantities are computed directly from the lengths of chains and the structure of loops, without any game tree search, and they guide whether to keep or give up control in borderline cases [2].

Unsafe moves. An unsafe move is any edge that allows the opponent to complete a box immediately on their next turn. Our heuristic agent penalises unsafe moves at one ply, which mirrors practical guidance for human play [7].

Two ply lookahead. Two ply lookahead evaluates each legal move with a single opponent reply. Deep2 uses two ply specifically to assess whether a move hands the opponent a favourable handout or gives up control, rather than to perform wide search [6].

1.4 Literature Review

Although the rules are minimal, the depth of Dots and Boxes emerges from chains, loops, and sacrifices, topics studied extensively in combinatorial game theory and AI [1].

Combinatorial foundations. Berlekamp, Conway, and Guy develop an endgame theory built on components such as chains and loops, introduce control concepts such as the long chain rule, and relate positions to the Nimstring viewpoint where component values and parity guide optimal play [1]. They show why greedy captures can be inferior to controlled handouts. By conceding two boxes in a long chain, a player can maintain control and decide the order in which remaining components are opened. Although Dots and Boxes is a scoring game rather than a normal play impartial game, the Nimstring abstraction and component decomposition provide the conceptual bridge to Sprague–Grundy reasoning. This component first perspective underpins the chain aware heuristic here: detect chains and loops and reason about control and parity so decisions stay interpretable rather than opaque search scores.

Endgame algorithms. Building on that perspective, Allcock gives simple algorithms for endgames consisting entirely of loops and long chains; both the victory margin and an optimal move follow from the structure of the current position without any game tree search [2]. Key quantities such as controlled value and terminal bonuses arise from the arrangement and parity of components. This motivates component wise evaluation. Identify chains and loops early and prefer moves that preserve

control without needing a heavy endgame solver, and it fits naturally with experiments on 4×4 , 5×5 , and 6×6 boards while not being tied to a specific size.

Human usable heuristics. Buzzard and Ciere analyse loony endgames, positions where naively opening the only chain is costly, and give simple rules such as the two box handout that achieve near optimal results in those settings without deep search [7]. That philosophy informs the one ply safety checks in our heuristic agent, which explicitly avoids moves that hand the opponent immediate captures and thus approximates strong play with minimal computation.

Search heavy agents and limits. Classic minimax with hand crafted evaluations such as box counts and chain features struggles as board size grows because of branching and horizon effects. Monte Carlo Tree Search studies show vanilla rollouts underperform unless tailored to chain logic, and large simulation budgets are needed to approach strong play [8]. Rather than hope search discovers chain structure, the present system encodes chain knowledge directly via interpretable heuristics.

Learning based agents. Reinforcement learning can train policies on small boards such as 3×3 and achieve reasonable win rates against baselines, but learned policies are typically opaque and often do not transfer to larger boards without retraining. Reward shaping can also bias agents toward always chaining regardless of control [9]. These transfer and interpretability limits argue for transparent, domain informed heuristics whose behaviour can be explained, debugged, and carried across board sizes without retraining.

Selective lookahead with heuristics. Recent work couples domain heuristics with shallow selective lookahead. Niu’s McGill Directed Reading Program report outlines a strategy that combines long chain reasoning with two ply lookahead, discussing when to double cross to retain control and demonstrating competitive play on modest boards with far less computation than full search [6]. In the same spirit, Deep2 - our two-ply heuristic agent - (see Section 2) combines linear time chain and loop identification with targeted two ply lookahead focused on control transitions and double cross risk.

Despite rich theory and AI work, cross platform implementations that integrate domain heuristics are still limited. Many mobile and web clones rely on generic tabletop engines or simple opponents. This work instead adopts cross platform engineering while integrating chain aware heuristics and a reproducible evaluation pipeline, bridging theory and practice [3], [4]. A modular Dart engine and Flutter UI deployable to Android, Web, and Desktop are coupled includes three AI agents: Random, Heuristic1, and Deep2, combining the theoretical grounding of [1] with the efficiency of selective lookahead [6] to create a practical platform for comparative experimentation in strategic AI for combinatorial games. Automated evaluation across multiple board sizes then examines behaviour

and trade offs without committing to heavy midgame search.

1.5 Objectives and Contributions

1.5.1 Objectives

This work has four goals that address gaps in current implementations and AI studies. First, create a cross platform version of Dots and Boxes from a single Dart and Flutter codebase so behaviour is consistent across Android, Web, and Desktop [3], [4]. Second, implement a modular engine that cleanly separates UI from rules and is structured to generalise beyond two players, providing a stable foundation for AI development. Third, implement three agents with increasing strategic sophistication: a uniformly random baseline; a heuristic agent that avoids unsafe moves that hand immediate captures to the opponent; and a chain aware agent that detects chains and loops, applies the long chain rule, uses selective two ply lookahead to avoid enabling double crosses, and sacrifices only when it preserves control [1], [2]. Finally, design and run an evaluation pipeline across board sizes using automated simulation and statistical analysis for reproducible comparisons.

1.5.2 Contributions

We built an extensible Dart engine that supports two players, designed to generalise further, with legal move generation, box detection, turn control, and deep state cloning for simulation. The engine is UI agnostic, so the same state model serves human play, AI play, and automated experiments. The board uses separate horizontal and vertical edge matrices to make box completion and ownership checks efficient.

Our AI implementation includes three agents: Random, Heuristic1 and Deep2. Random chooses from legally available moves uniformly at random, providing a baseline. Heuristic1 performs a one ply safety analysis to identify and avoid edges that give the opponent an immediate box, reflecting human usable endgame guidance [7]. Deep2 is a chain aware two ply heuristic: it detects chains and loops (internal boxes with two open edges, endpoints with one), applies long chain control, evaluates double cross risk by simulating the opponent's best reply, and permits strategic sacrifices only when net control is preserved [1], [2]. This targeted two ply mirrors expert reasoning by assessing the opponent's reply before committing to a move.

The Flutter UI supports local two player games, single player matches against all three AI variants, and online multiplayer using Firebase Realtime Database. A shared room model keeps guest and host in sync via realtime updates [5].

For evaluation, a Dart benchmarking harness simulates about 300 automated games per pairing across 4×4 , 5×5 , and 6×6 grids, exporting metrics such as win rate, unsafe move counts, turn statistics, and largest scoring streaks to CSV for Python analysis. The broader question is whether interpretable heuristics, grounded in expert knowledge, can compete in Dots and Boxes while staying fast enough for phones and web deployment. The architecture is intentionally extensible so that new agents or evaluation protocols, for example ablations or human in the loop studies, can be added without refactoring existing modules.

2 Methodology

2.1 Design and Technology Selection

Rationale for the technology stack

We use Flutter and Dart so a single codebase compiles to native binaries for mobile and desktop, and to the web via JavaScript or WebAssembly (Wasm). Keeping one rules engine across targets preserves experimental reproducibility and avoids platform drift. [3], [4]. React Native or pure native approaches would require bridges and multiple codebases, increasing maintenance and the chance of platform specific behaviour that confounds AI evaluation. For online play we rely on Firebase Realtime Database because it provides real time listeners, reconnection, and offline queueing out of the box, which lets us focus on agents and game logic rather than operating a bespoke backend [5].

Rationale for the AI approach

Our approach is deliberately simple: encode the chain/loop structure that matters and look only a move or two ahead. Concretely, we use three interpretable agents: *Random*, *Heuristic1*, and *Deep2* - each for a distinct role: *Random* is a uniform baseline; *Heuristic1* simulates one reply and avoids handing over a box; and *Deep2* looks two steps ahead - it tries our move, then the opponent's strongest immediate reply, and prefers moves that still leave us better off. These agents are fixed-depth, run entirely on-device, and are described in 2.4.

Our aim is interpretability and efficiency on resource constrained devices. Deep minimax and large Monte Carlo budgets are slow on phones and make the reason for a move opaque; generic rollouts also struggle to recognise and exploit chain structure without bespoke policies [8]. We therefore emphasise domain informed heuristics and shallow lookahead that encode expert ideas such as chain control and the hard hearted handout [1], [2], [6]. Black box learning approaches can achieve

good win rates on small boards but often fail to transfer without retraining and do not explain their decisions [9]. The question here is how far transparent heuristics can go when engineered into a modular system.

2.2 System Architecture

At a high level (Fig. 4), we keep one source of truth - the game engine state and put everything else around it. The Flutter UI only renders and collects input. A small orchestrator decides whose turn it is and either forwards a human edge or asks an agent for one. An online client mirrors moves via Firebase Realtime Database so two devices replay the exact same sequence and end up in the same state [3]–[5]. The same engine powers local human play, human-agent games, online play, and automated tournaments. The rest of this subsection walks through how the pieces fit together.

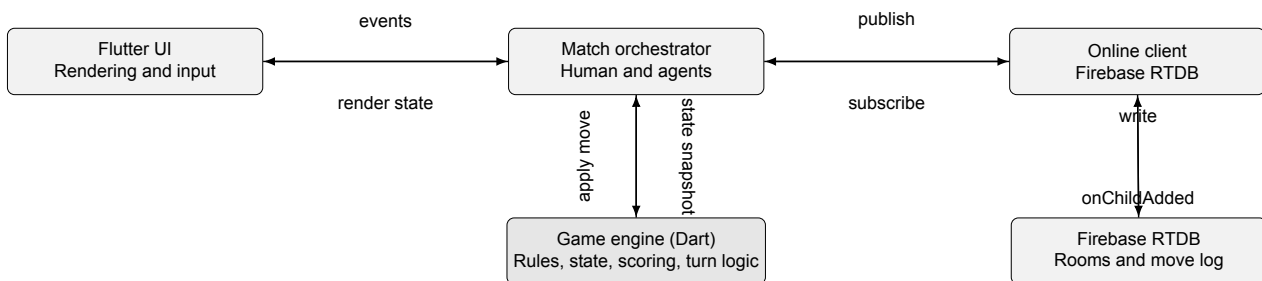


Fig. 4. Clean architecture. The engine owns rules and state transitions; the UI renders and collects input. The online client only transports moves via RTDB so both engines replay the same sequence [3]–[5].

The engine is pure: no UI code, no networking, no timers, no randomness. It exposes a serialisable state and a small set of operations on that state. A move is just the pair of adjacent dots that define an unused edge. Applying a legal move updates the edge grids and any completed boxes; if a box is claimed the same actor moves again, otherwise the turn passes. Invariants are simple and enforceable: edges are never reused; box ownership never changes after assignment; the next actor is uniquely determined by the state. Because the same input produces the same next state on every platform, we can clone state cheaply for simulation and trust that two devices will stay in lock-step.

Around that core, the orchestrator runs the match loop. It reads the current snapshot from the engine, asks the UI for a human edge or calls the agent if it’s an agent’s turn, applies the chosen edge in the engine, and redraws. If the move scored a box, the same actor acts again; if not, turn control flips. When a game is online, the orchestrator also publishes the move to the room so the peer can replay it.

Online transport is kept minimal. A room stores player identities, a started flag, and an append-only move log. When a move arrives from Firebase, the receiver sets the engine’s actor to the recorded index and replays that edge. Ordering is the insertion order of the log; replay is idempotent because

the engine refuses already-used edges; on reconnect, queued writes are delivered in order and both devices converge [5]. The result is lock-step play with identical state on both devices (see also Fig. 5).

Internally the engine represents an $n \times n$ board with two boolean grids for horizontal and vertical edges of sizes $(n+1) \times n$ and $n \times (n+1)$, plus a grid of owners for the $n \times n$ boxes. This layout gives $O(1)$ legality checks, local adjacency tests for box completion, and inexpensive full-state cloning for lookahead. Because the structure is serialisable and platform-agnostic, the exact same state flows through UI, agents, and online layers unchanged.

2.3 Real Time Synchronisation and Multiplayer

This subsection explains how two devices stay in sync during online play, then shows the message flow in Fig. 5. We use a single shared room in Firebase Realtime Database that holds player metadata, a *started* flag, and an append-only *moves* list. Each move records the ordered pair of endpoints and the actor index. Both devices subscribe to the room. When a move arrives, the receiver sets the engine's actor to the recorded index and calls `playEdge`. Because the engine produces the same next state for the same input on every platform, both engines remain identical after each replay [5].

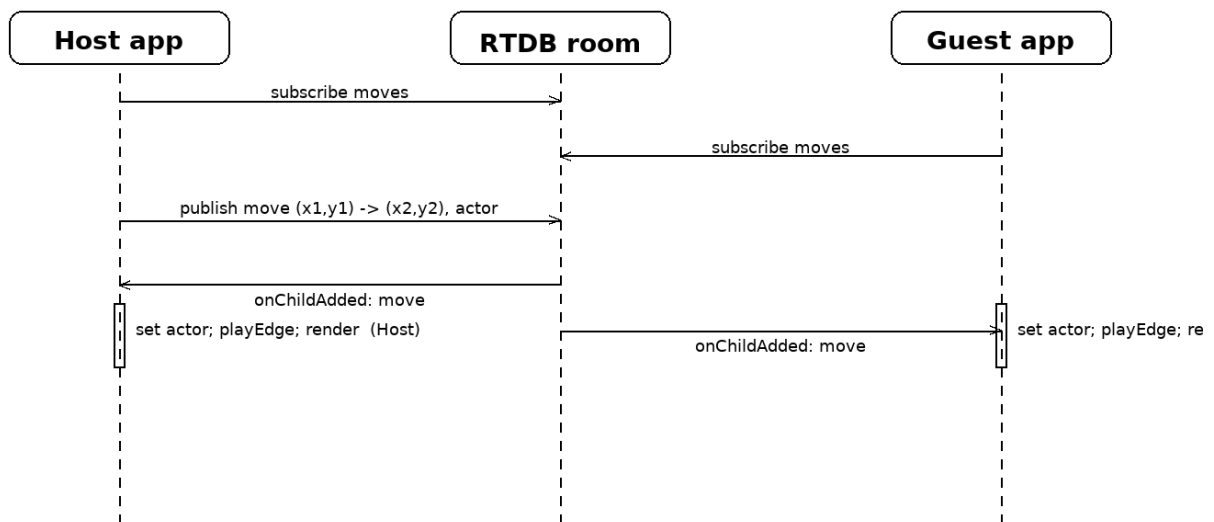


Fig. 5. Move propagation. One published entry is replayed by both engines, keeping state identical across devices and through disconnects [5].

Two simple guarantees keep the stream well-behaved. First, **ordering**: the append-only list defines replay order. Second, **idempotence**: the engine refuses already-used edges, so reapplying the same move does nothing and state cannot drift. These two, together with the SDK's **reconnect**

behaviour, are enough for robustness: on reconnect, queued writes are delivered in insertion order, both devices replay anything they missed, and the states match again [5].

Turn handling is enforced at both ends. Locally, the orchestrator only publishes when the local actor matches the engine’s current index. Remotely, the receiver still sets the actor from the move record before replaying it. This “actor-in-record” rule keeps the stream authoritative without a separate lock service: whoever the record says moved is treated as the mover, and the engine’s turn logic follows from there.

2.4 AI Agent Designs

We implement three agents that operate directly on the engine state and return one legal edge. They are designed to be interpretable and to run quickly on mobile hardware, in line with practical endgame advice [1], [7] and with prior findings that generic deep search on Dots and Boxes struggles without domain policies [8].

Notation used in pseudocode

- **G**: serialisable engine state (board + edges + owners).
- **e**: a candidate edge (ordered pair of adjacent dots).
- **LEGAL_EDGES(*G*)**: list of legal edges in state *G*.
- **CLONE(*G*)**: deep copy of *G*.
- **APPLY(*G*, *e*)**: apply edge *e* to *G* (updates boxes/turn).
- **TURN(*G*)**: index of the player to act in *G*.
- **SCORE(*G*, *i*)**: box count for player *i* in *G*.
- **OTHER(*i*)**: $1 - i$ for two-player games.
- **RAND_CHOICE($\cdot$)**: uniform random tie-breaker over a list.
- **M**: a set/list of legal edges (e.g., $M \leftarrow \text{LEGAL_EDGES}(G)$).
- **S**, **S1**, **S2**: cloned states used for simulation.
- **r**: a reply edge in inner simulation loops.
- **nxt**: actor index after applying a candidate (from **TURN(*S*)**).

- **base**: baseline score for $next$ in S (used in Heuristic1).
- **me, you**: current mover and opponent indices.
- **baseMe, baseYou**: baseline scores in the original G (used in Deep2).
- **ALL, SAFE, POOL**: working lists in Heuristic1 (all candidates; candidates that do not hand over an immediate box; final choice pool).

Random agent. *Random agent* selects uniformly from the legal set. It is a pure baseline that exercises legality, scoring, and turn rotation and provides a lower bound for strength and a reference for timing. The implementation in `randomMove` enumerates legal edges once and chooses one uniformly at random.

```
function pick_move_RANDOM( $G$ )  $\rightarrow e$ 
   $M \leftarrow \text{LEGAL\_EDGES}(G)$  return RAND_CHOICE( $M$ )
```

Heuristic1. *Heuristic1* avoids candidates that let the next actor in the simulated position take a box in a single reply. The helper `givesBoxToOpponent` clones the state, applies the candidate, queries the engine for who moves next in that position, then checks all one-edge replies for a score increase for that same actor. If any reply scores, the candidate is marked unsafe and avoided when possible. This mirrors the simple over-the-board rule “do not hand over a box immediately.” A nuance of the current baseline is that the “next actor” is whatever the engine sets after the candidate. If the candidate completes a box and the mover keeps the turn, the procedure still tests the mover’s one-edge replies, which can mark some good capturing moves as unsafe. This is exactly how the published `givesBoxToOpponent` and `heuristic1Move` behave in the code.

```
function givesBoxToOpponent( $G, m$ )  $\rightarrow \text{bool}$ 
   $S \leftarrow \text{CLONE}(G)$ ;  $\text{APPLY}(S, m)$   $next \leftarrow \text{TURN}(S)$ ;  $base \leftarrow \text{SCORE}(S, next)$  foreach
     $r \in \text{LEGAL\_EDGES}(S)$  do
       $S2 \leftarrow \text{CLONE}(S)$ ;  $\text{APPLY}(S2, r)$  if  $\text{SCORE}(S2, next) > base$  then
        return true
  return false
```

```
function pick_move_HEURISTIC1( $G$ )  $\rightarrow e$ 
   $ALL \leftarrow \text{LEGAL\_EDGES}(G)$   $SAFE \leftarrow [m \in ALL \mid \neg \text{givesBoxToOpponent}(G, m)]$ 
   $POOL \leftarrow \text{if } (SAFE \neq \emptyset) \text{ then } SAFE \text{ else } ALL$  return RAND_CHOICE( $POOL$ )
```

Deep2. *Deep2* evaluates each legal move with a two-ply value: our immediate gain after playing the move on a clone, minus the opponent’s *best immediate* gain over all single-edge replies from that resulting position. In the current library implementation (`deep2Move`) the baselines for both gains are taken from the live state before the move, that is, `baseMe` and `baseYou` are computed once from G and reused. This keeps the routine compact and fast and discourages enabling large one-move rejoinders, while deliberately avoiding explicit chain detection, parity analysis, or multi-step “keep the turn” modelling [1]. It is an interpretable, small minimax-style lookahead suitable for real-time play.

```

function pick_move_DEEP2( $G$ )  $\rightarrow e$ 
   $me \leftarrow \text{TURN}(G); you \leftarrow \text{OTHER}(me)$   $baseMe \leftarrow \text{SCORE}(G, me); baseYou \leftarrow \text{SCORE}(G, you)$ 
   $bestVal \leftarrow -\infty; bestMoves \leftarrow []$  foreach  $m \in \text{LEGAL\_EDGES}(G)$  do
     $S1 \leftarrow \text{CLONE}(G); \text{APPLY}(S1, m)$   $meGain \leftarrow \text{SCORE}(S1, me) - baseMe$   $worstYou \leftarrow 0$ 
    foreach  $r \in \text{LEGAL\_EDGES}(S1)$  do
       $S2 \leftarrow \text{CLONE}(S1); \text{APPLY}(S2, r)$   $youGain \leftarrow \text{SCORE}(S2, you) - baseYou$  if
         $youGain > worstYou$  then
           $worstYou \leftarrow youGain$ 
     $val \leftarrow meGain - worstYou$  if  $val > bestVal$  then  $bestVal \leftarrow val; bestMoves \leftarrow [m]$ 
    else if  $val = bestVal$  then  $bestMoves.push(m)$ 
  return  $\text{RAND\_CHOICE}(bestMoves)$ 

```

Overall, Random agent provides a reproducible lower bound and timing reference. *Heuristic1* encodes a transparent “do not hand over an immediate box” policy by checking next-actor replies in a cloned state, exactly as in the codepath `givesBoxToOpponent  $\rightarrow$  heuristic1Move`. *Deep2* performs a compact two-ply lookahead that subtracts the opponent’s best single-edge rejoinder from our immediate gain, following the library routine `deep2Move`. Together these agents let us compare simple, interpretable baselines and a shallow lookahead within a single modular engine.

2.5 Cross-platform testing and validation

We verified parity between Android and web by executing the same scripted traces on both targets and checking that state snapshots and scores match after each move. Rendering and input were tested on devices and in desktop browsers. For online play we exercised publish, fan out, and replay under normal connectivity and under forced disconnects to confirm that queued writes are delivered and both engines converge [3]–[5]. Timing and head-to-head results are reported in the Evaluation chapter.

3 Evaluation and Reflection

3.1 Experimental Design

The purpose here is straightforward: measure how *Random*, *Heuristic1*, and *Deep2* behave across board sizes using the exact rules the app runs. We simulate engine-level matches on 4×4 , 5×5 , and 6×6 boards. For each grid size, every pairing is played in both orders (A as P1 vs B, and B as P1 vs A), and we also run self-play for each agent. We use $n = 300$ games per ordered pairing per grid. All results in this chapter are engine-only; there is no UI or networking in the loop, so the numbers reflect decision logic rather than rendering or transport.

The evaluation uses metrics that match expert advice on Dots and Boxes endgames: win rate as the final outcome; unsafe-move counts as a measure of “third-side” discipline; longest scoring streak as a pragmatic proxy for chain harvesting; total turns as a simple indicator of tempo; and score margin as an overall payoff measure. Unsafe discipline and the timing of small hand-outs are central themes in the literature; the value is not in avoiding every unsafe, but in choosing the right moment to concede a little to keep control of the larger harvest [1], [7]. Longest streak is not a perfect chain detector, but on these board sizes it correlates well with the phenomenon we care about and is trivial to compute from game logs. Longest streak counts how many boxes a player takes in one turn. That is not the same as “was there a long chain.” It can look big if the opponent leaves lots of easy three-siders scattered (no real chain), and it can look small if good play splits a big harvest across two turns (all-but-two / all-but-four). Loops can also add to or break a run depending on parity. Still, on 4×4 – 6×6 boards, strong chain control usually shows up as a long one-turn run, so across many games the agent that manages chains well tends to have higher longest-streak values.

3.2 Data Pipeline and Validation

The benchmark runner produces a per-game CSV with, for each match, the grid size, the identity of P1 and P2 agents, scores, counts of flagged unsafe moves per side, turns, and the longest scoring streak per player. Before analysis, we check that replaying the ordered pairings from the per-game CSV reproduces any aggregate tables. Where minor mismatches appear in convenience summaries, we treat the per-game CSV as the authoritative match log for this project and fix the summary. All plots and tables use the CSV directly to avoid silent aggregation errors.

3.3 Metric Definitions and Rationale

For each agent pair and board size, we fix who starts (P1) and play n games in that order. The *ordered P1 win rate* is the fraction of those games won by the starter. We also report the reverse order (swap who starts). We focus on P1 because, for a fixed order, the P2 rate is $1 - P1$, and because each agent appears as P1 in the opposite order; the two ordered P1 rates together show both strength and any start advantage without duplicating columns.

Unsafe moves are counted by simulating a one-move reply after a candidate and checking for an immediate score increase for the next actor; this matches how `givesBoxToOpponent` is implemented in the code and reflects the well-known “don’t hand over a box immediately” heuristic. Longest scoring streak captures whether a player engineered a sustained run of boxes in a single turn, the expected payoff from good chain management [1]. Turn count and score margin give tempo and payoff context.

3.4 Results and Visualisation

The ordered head-to-head heatmaps for 4×4 , 5×5 , and 6×6 (Figs. 6–8) show P1 win rates per pairing; *lighter* cells indicate a stronger P1 advantage. The ranking stays stable across board sizes: **Deep2 > Heuristic1 > Random**. Deep2’s advantage persists as grids grow because its two-ply value accounts for the opponent’s best immediate reply; the policy avoids giving away obvious multi-box rejoinders and steers into sequences where the next long run remains under its control [1], [7]. Heuristic1 suppresses obvious third-side blunders and therefore maintains a steady edge over Random, but without looking a move ahead it cannot protect itself from the simple rejoinders that Deep2 explicitly prices into its choice. Self-play cells sit near 50% for Deep2 and Heuristic1, as expected for symmetric agents; the Random self-play shows a notable second-player edge on 6×6 .

Concrete readings from the heatmaps.

- **4×4:** Deep2 vs Heuristic1 — Deep2 starts: **100.0%** (300/300); Heuristic1 starts: **0.0%** (0/300). Deep2 vs Random — **100.0%** and **0.0%**. Heuristic1 vs Random — H1 starts: **55.7%** (167/300); Random starts: **44.7%** (134/300). Self-play P1 rates: Deep2 **52.3%**, Heuristic1 **52.0%**, Random **50.0%**. (Figs. 6–8)
- **5×5:** Deep2 vs Heuristic1 — **100.0%** and **0.0%**. Deep2 vs Random — **100.0%** and **0.0%**. Heuristic1 vs Random — H1 starts: **57.0%** (171/300); Random starts: **49.3%** (148/300). Self-play: Deep2 **46.7%**, Heuristic1 **56.3%**, Random **50.3%**.

- **6×6:** Deep2 vs Heuristic1 — **100.0%** and **0.0%**. Deep2 vs Random — **100.0%** and **0.0%**.
Heuristic1 vs Random — H1 starts: **56.7%** (170/300); Random starts: **42.3%** (127/300). Self-play: Deep2 **51.0%**, Heuristic1 **51.0%**, Random **41.7%**.

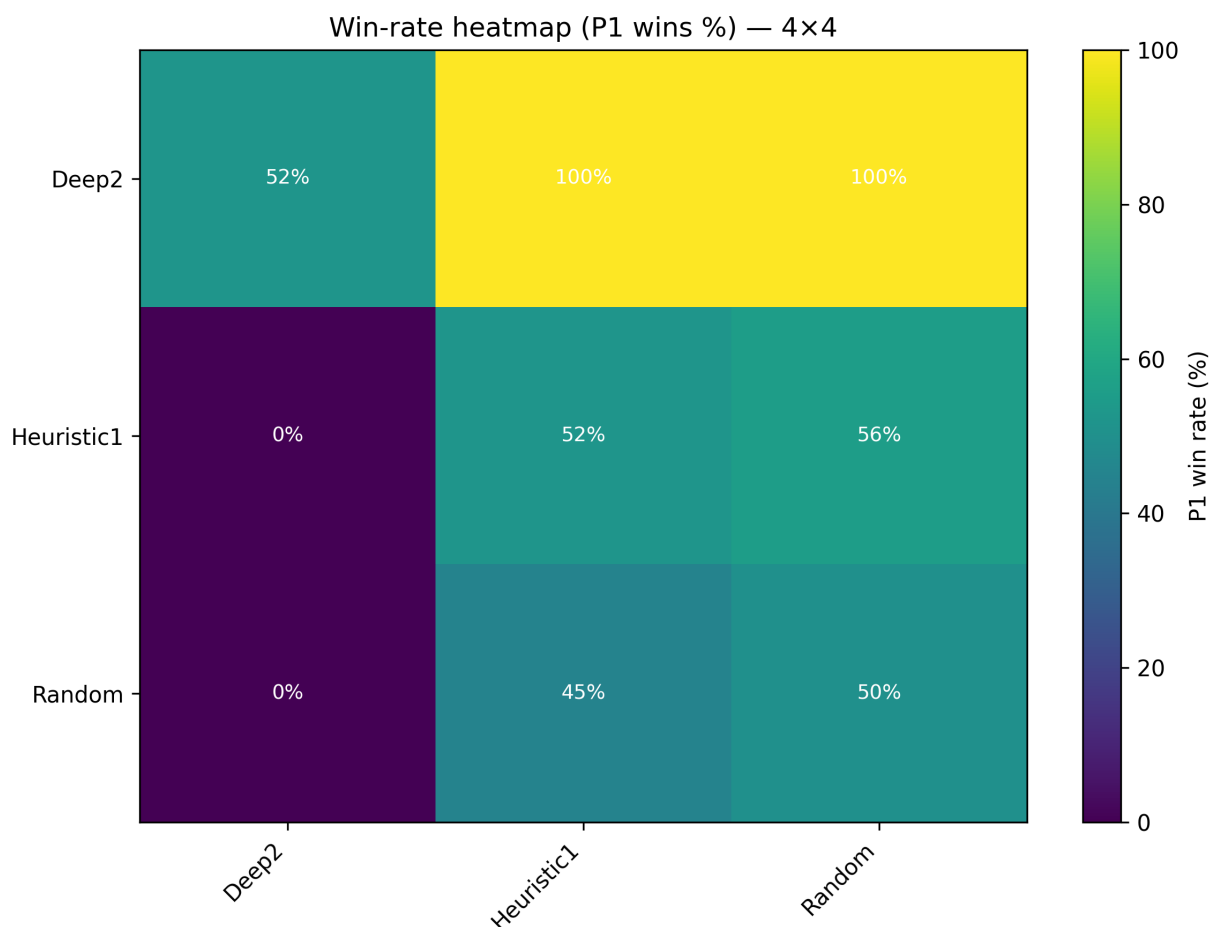


Fig. 6. Head-to-head P1 win-rate heatmap on 4×4 (lighter = stronger P1). **Reading:** Deep2 dominates both Heuristic1 and Random regardless of who starts; Heuristic1 has a modest start-sensitive edge over Random.

Unsafe moves, streaks, margin, and tempo. Per-agent aggregates (mean unsafe / mean longest streak / mean score margin / median turns) make the trade-offs clear:

- **4×4** — Deep2 **10.8 / 7.9 / +9.5 / 40**; Heuristic1 **7.3 / 3.9 / -4.1 / 40**; Random **10.8 / 3.6 / -5.3 / 40**. Heuristic1 is safer but gives up margin; Deep2 converts a few extra risks into long runs and a positive margin. (Figs. 9, 10)
- **5×5** — Deep2 **17.3 / 10.2 / +15.2 / 60**; Heuristic1 **11.5 / 5.0 / -6.9 / 60**; Random **17.0 / 4.7 / -8.3 / 60**. As boards grow, Deep2’s runs lengthen and margins widen.
- **6×6** — Deep2 **25.3 / 12.6 / +22.4 / 84**; Heuristic1 **16.7 / 6.3 / -10.2 / 84**; Random **24.4 / 5.8 / -12.3 / 84**. Deep2’s “small hand-out, big harvest” pattern is strongest here.

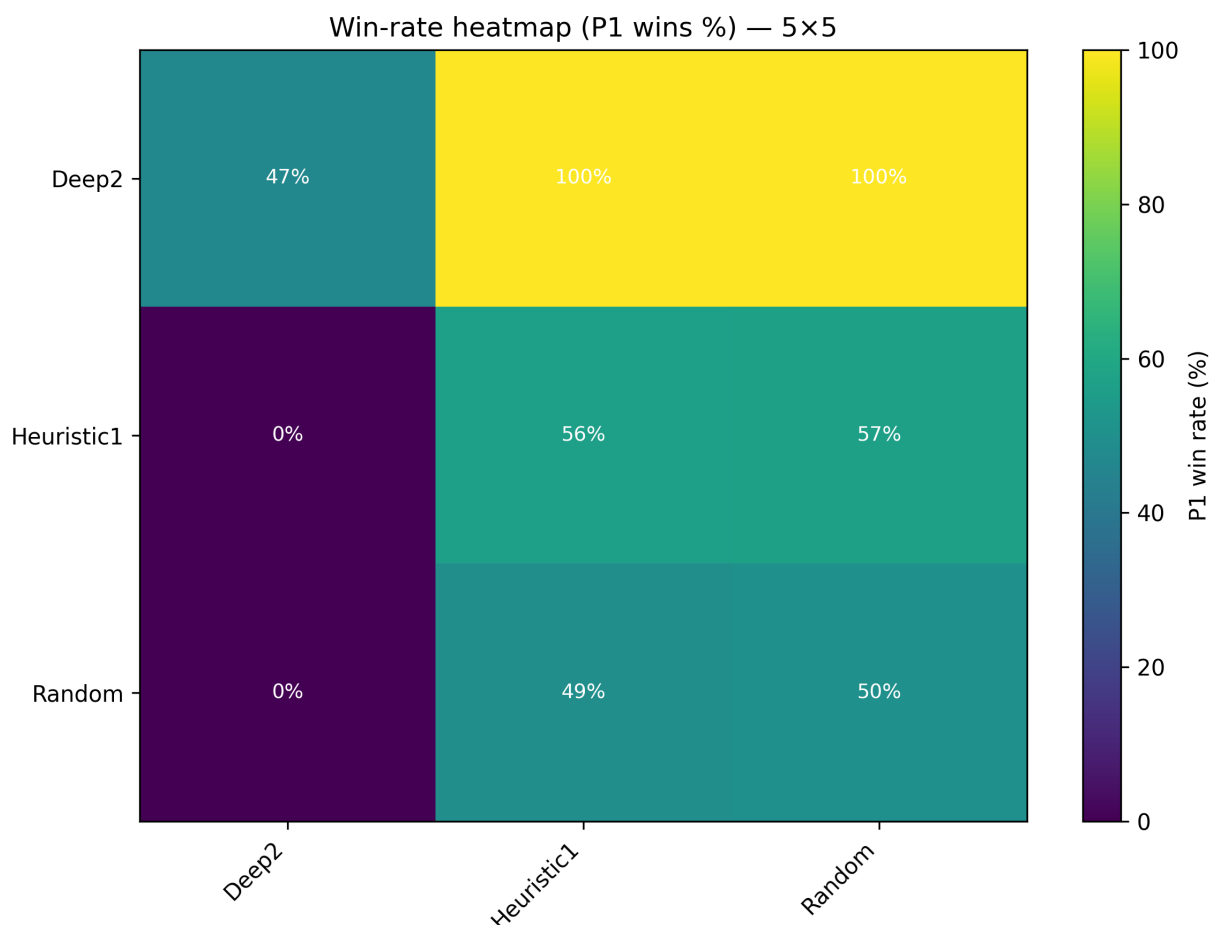


Fig. 7. Head-to-head P1 win-rate heatmap on 5×5 (lighter = stronger P1). **Reading:** Same ordering; start effects visible mainly in Heuristic1 vs Random.

Chain exploitation appears in the longest scoring streaks. We do not perform explicit chain or loop detection in this project; instead, we use longest streak as a pragmatic proxy. On these board sizes, genuine chain control typically manifests as a sustained run of captures in a single turn [1]. The plot shows Deep2 with the highest means and the steepest growth as grids enlarge (**7.9** → **10.2** → **12.6**), while Heuristic1 (**3.9** → **5.0** → **6.3**) and Random (**3.6** → **4.7** → **5.8**) stay lower. That is exactly what a “sacrifice now, harvest later” policy ought to produce. (Fig. 11)

A compact risk–reward view makes the timing trade-off visible at a glance. Plotting score margin against unsafe differential (P1 unsafe minus P2 unsafe) per game, the Deep2 cloud concentrates where controlled risk buys margin, Heuristic1 clusters near low-risk/low-reward, and Random spreads broadly. This pattern matches the theory: avoid third sides unless a small hand-out secures the long chain [1], [7]. Even without the scatter, the same reading follows from the unsafe and streak plots together.

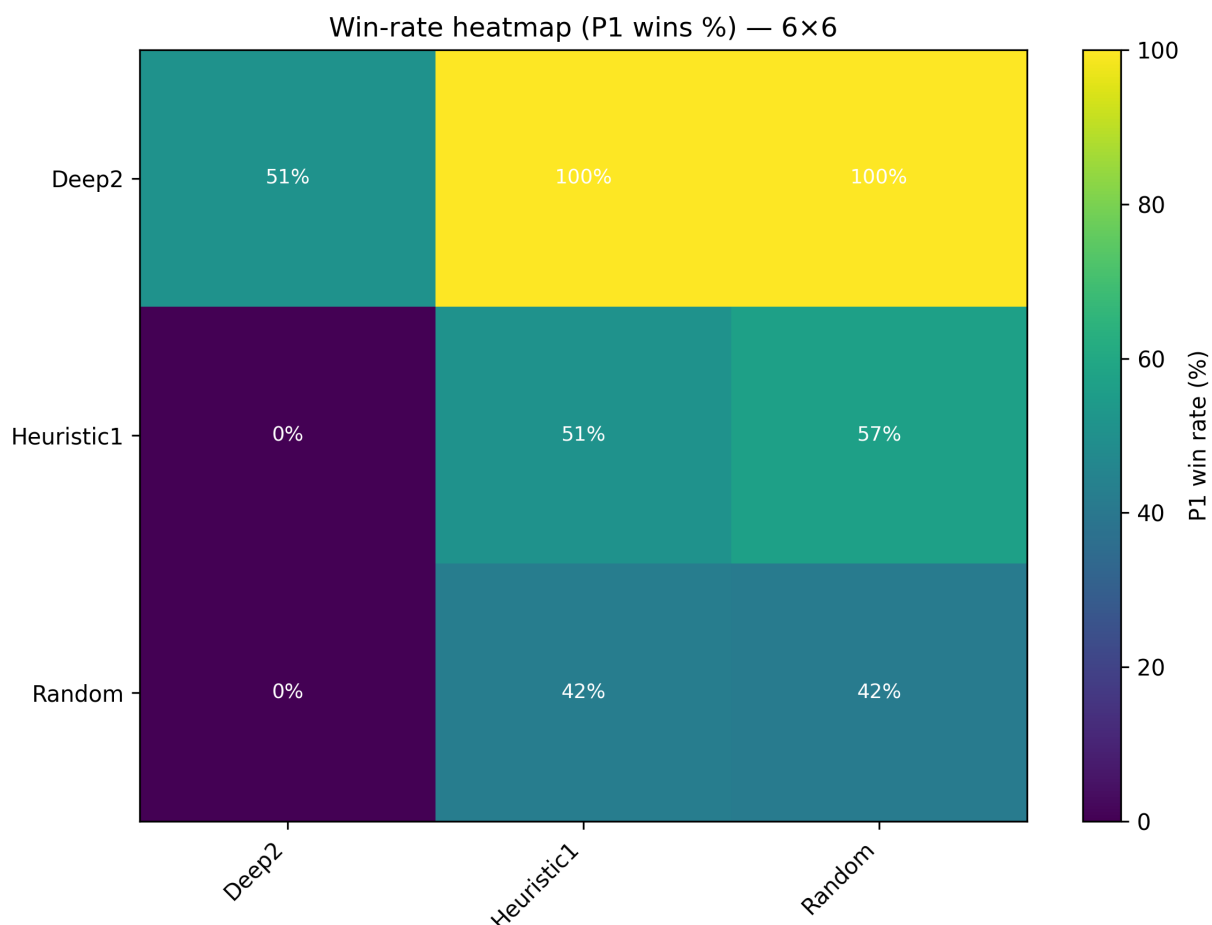


Fig. 8. Head-to-head P1 win-rate heatmap on 6×6 (lighter = stronger P1). **Reading:** Deep2 remains start-insensitive; Random self-play shows a second-player edge.

3.5 Per-Grid Narrative

On 4×4, simple safety rules already move the needle: Heuristic1 edges Random when it starts (55.7%) and slips when it does not (44.7%), while Deep2 dominates both ways. On 5×5, streak means separate more decisively (Deep2 $\approx$ 10.2 vs Heuristic1 5.0), and margins widen. By 6×6, the difference between “avoid third sides” and “time the hand-out” is pronounced: Deep2’s longest streak mean reaches $\approx$ 12.6 and its average margin $\approx$ +22.4, with median turns $\approx$ 84 indicating efficient harvests.

3.6 Statistical Perspective

With $n = 300$ per ordered pairing, sampling error on win-rate estimates is small enough to separate mid-single-digit differences from noise. Reporting both orders with raw counts makes start effects visible without extra derived metrics. Because each agent appears as P1 in one order and as P2 in the other, the two ordered P1 rates are sufficient to compare strength and to expose start advantage.

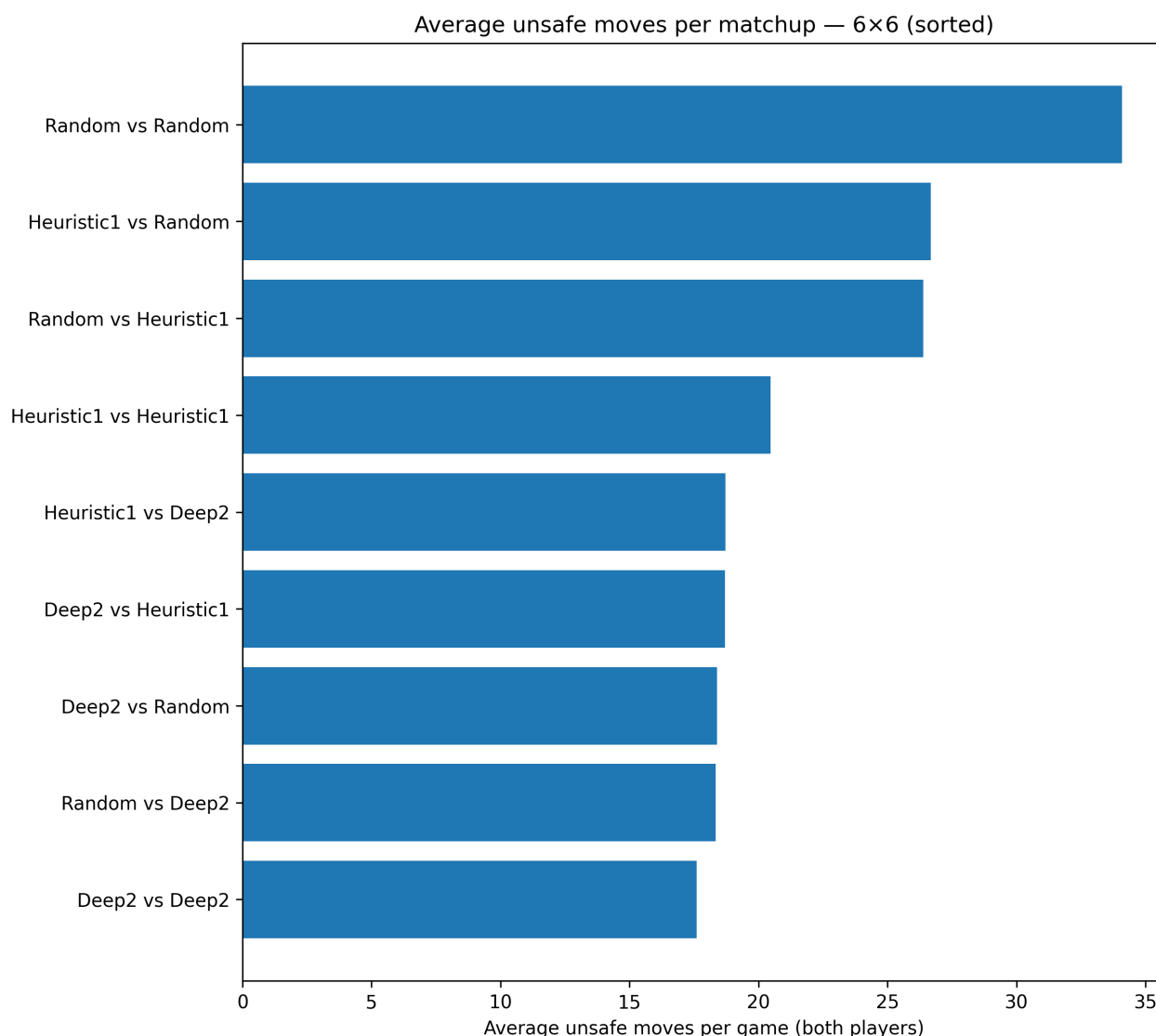


Fig. 9. 6×6 pairings ranked by combined unsafe moves per game (lower is safer). **Reading:** Heuristic1 is the most frugal; Deep2 takes a few more risks but earns the largest margins.

3.7 Turn Economy and Tempo

Total turn counts serve as a simple reading of tempo. Random-vs-Random tends to produce longer, swingy games as third sides appear everywhere. Heuristic1 shortens games modestly by suppressing immediate giveaways. Deep2 either ends a phase quickly with a short concession or accelerates the harvest; both pressures reduce wasted turns, which shows up as tighter turn distributions (median $\approx 40, 60, 84$ across grids) and larger score margins once the harvest starts. The numbers reinforce the same idea from a different angle: it is not the avoidance of risk that matters, but the timing of it.

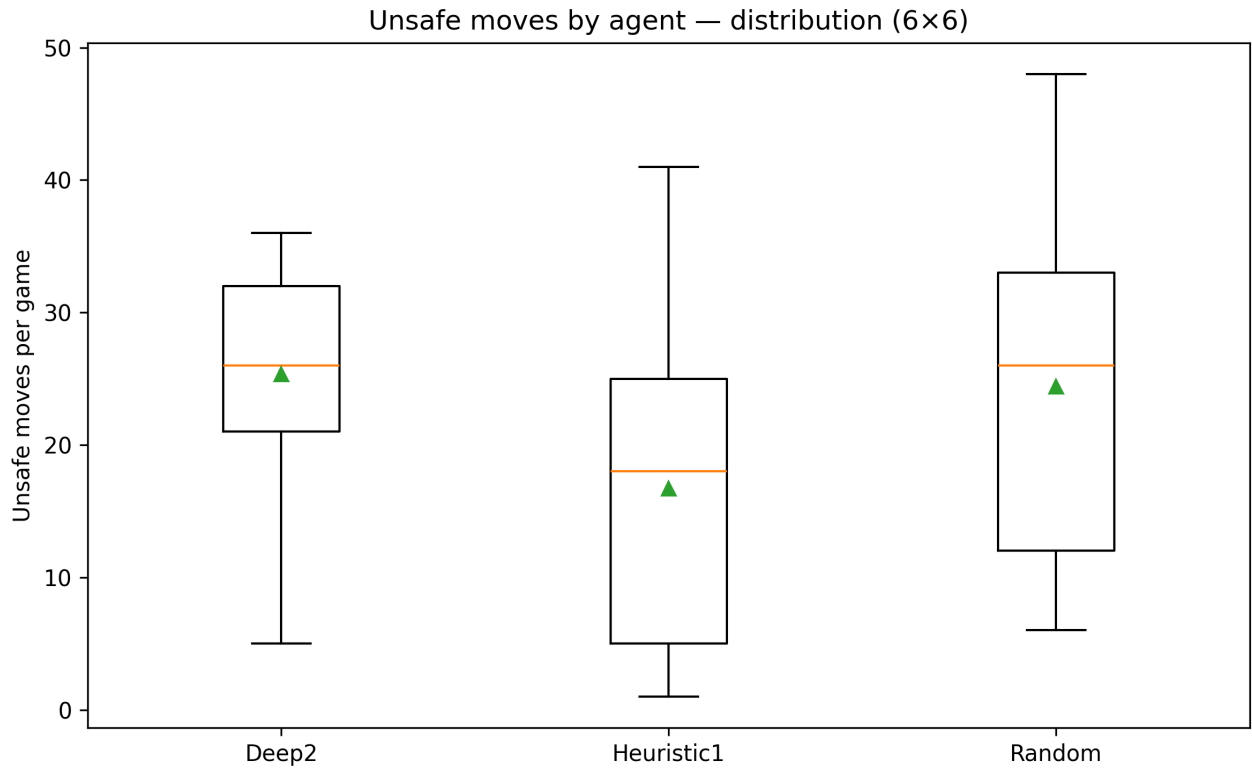


Fig. 10. Per-agent distribution of unsafe moves on 6×6 (median, IQR, whiskers; mean marked). **Reading:** Random is noisy; Deep2's risk level is controlled.

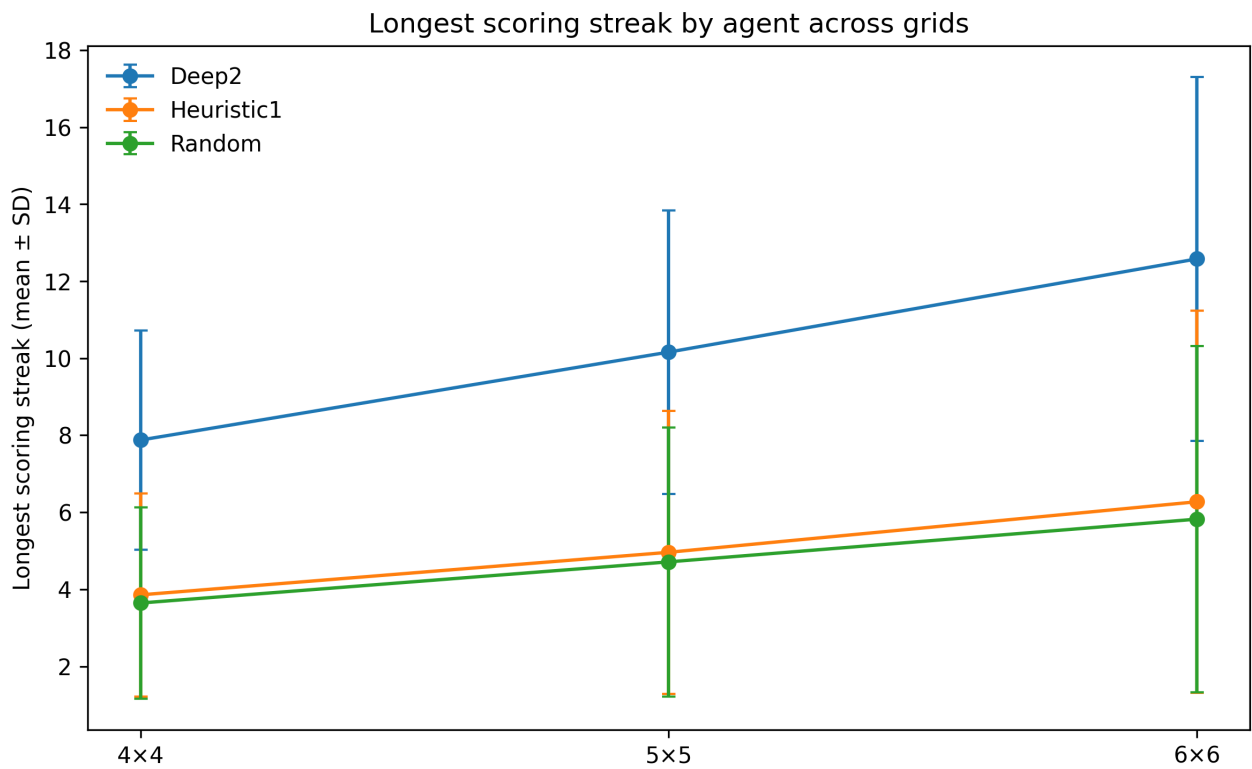


Fig. 11. Mean longest scoring streak by agent across grids (error bars show ± 1 standard deviation). **Reading:** Deep2 produces the longest single-turn runs as boards grow.

3.8 Performance and Product Context

During the runs we record move-selection time inside the engine. Medians are comfortably sub-millisecond to a few milliseconds on 4×4 – 6×6 . On 6×6 , the tail can exceed a single 60 Hz frame budget, but selection is not a per-frame operation and turns are discrete, so perceived responsiveness remains high. In the app, the same engine code path runs under Flutter; the online mode writes an append-only move log to Firebase Realtime Database, and both devices replay each entry after setting the recorded actor so they end in the same state [5]. Temporary disconnects converge as queued writes deliver. This is why the qualitative feedback on responsiveness and online stability lines up with the engine-only timing.

3.9 Product UI Overview

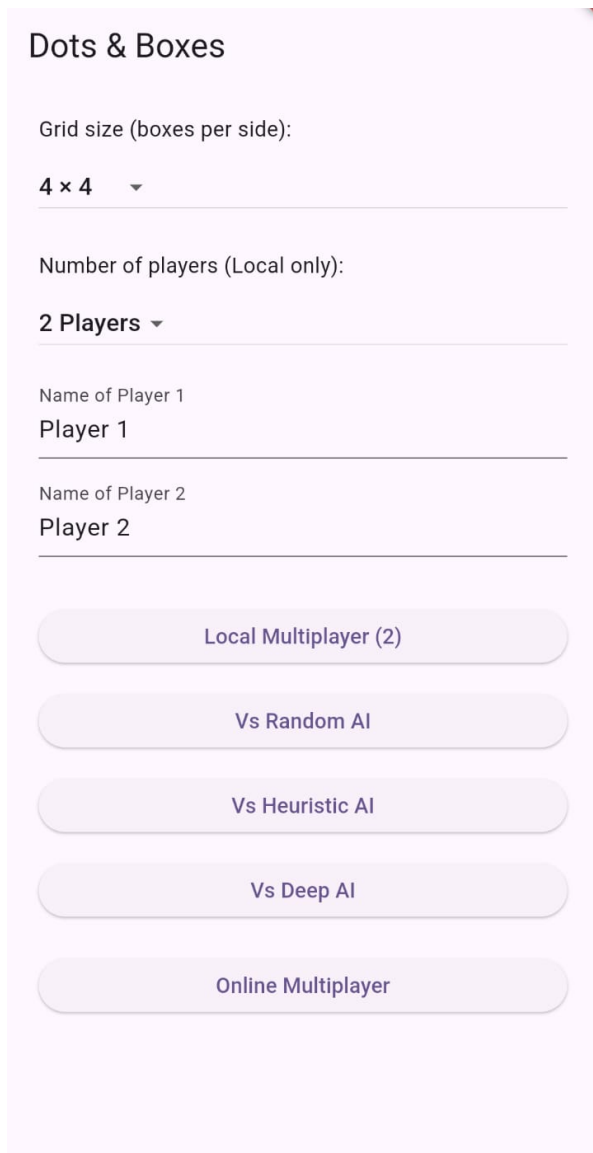
The start screen (Fig. 12 a) is the entry point for every evaluation path. Testers choose a grid size (4×4 , 5×5 , 6×6), set local player names, and launch a match with one tap. The mode buttons map directly to our agents and study conditions: *Local Multiplayer* (2) runs human vs human on one device; *Vs Random AI*, *Vs Heuristic AI*, and *Vs Deep AI* start human–agent matches with the Random, Heuristic1, and Deep2 policies; *Online Multiplayer* opens the room flow backed by Firebase. Keeping all choices on one screen made it easy to reproduce conditions during testing.

In game (Fig. 12 b and 12 c), the top bar shows the live score and whose turn it is. The board is a tappable dot grid; tapping an unused edge commits that edge. When the fourth side of a box is completed, the box fills with the capturing player’s colour and that player keeps the turn, so streaks appear as consecutive fills. A refresh icon resets the game; the back arrow returns to the start screen. The Android and Web screenshots demonstrate cross-platform parity: the same Dart engine applies rules, scoring, and turn logic on both targets.

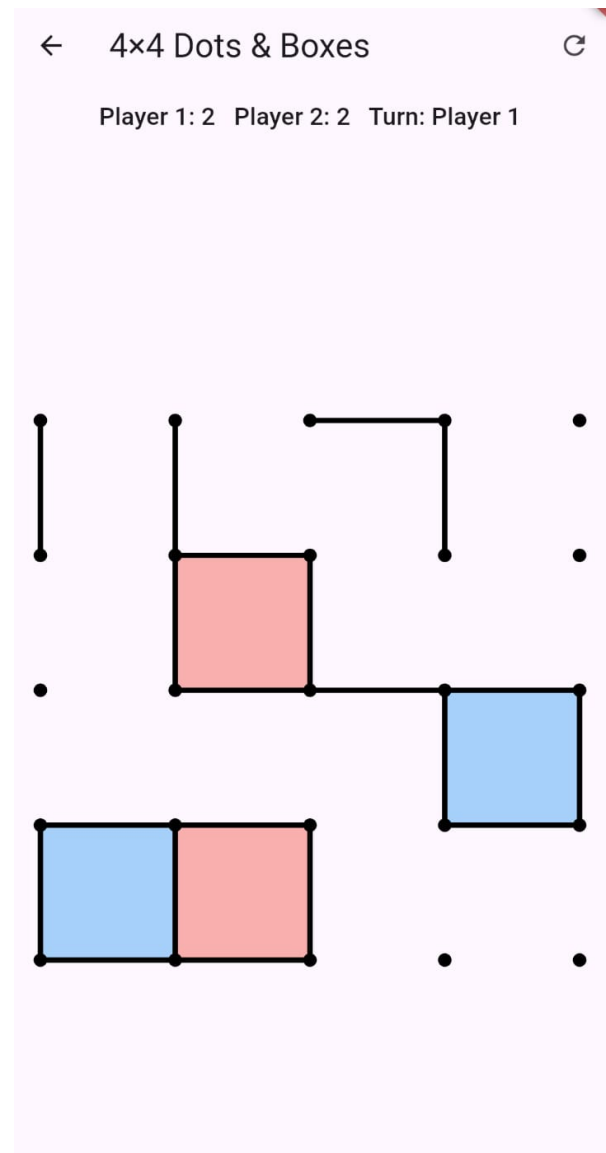
For online play, the host creates a room and shares a short room identifier; the guest enters that identifier to join. After a move is published, both devices receive the same record and replay it in the engine, which keeps state identical even if one side briefly disconnects.

3.10 User Experience Observations

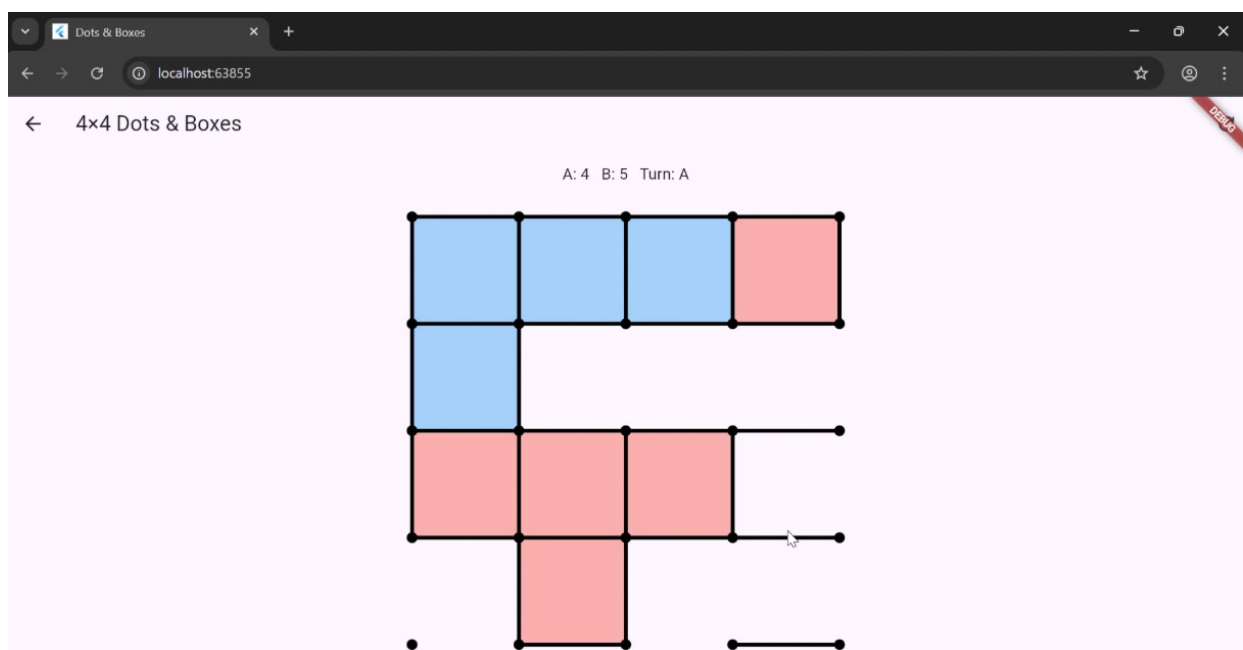
Feedback form responses and informal playtests show that turns feel immediate on mid-range phones and scoring is clear. The two most frequent requests were a shorter, simpler room ID flow and a more polished UI. Players still reported high enjoyment—both for online matches with friends



(a) Start screen: grid size, players, quick starts vs agents, online.



(b) In-game on Android: live score/turn; extra turn on capture.



(c) In-game on Web: same Dart engine, identical behaviour.

Fig. 12. Product snapshots used in evaluation and demos. The screenshots are from the same builds used during evaluation.

and for trying the three agents (Random, Heuristic1, Deep2) to compare styles. These are UI-side improvements; the engine and Realtime Database synchronisation ran reliably throughout.

3.11 Limitations

The study focuses on 4×4 – 6×6 boards. Larger boards may alter sacrifice timing and increase selection time for quadratic policies. Deep2 is a compact two-ply value with no deeper minimax/MCTS baselines in this project; generic rollouts are known to struggle on chain structure without bespoke policies [8]. Longest streak is a proxy for chain harvesting rather than explicit chain/loop detection; the proxy correlates well at these sizes but can overcount in runs of small opportunities. First-move effects exist; we report both orders with raw counts and read start effects directly from the heatmaps.

3.12 Lessons and Implications

The results support a simple claim: domain-aware heuristics with shallow lookahead can play strongly on phones, and their choices are easy to explain. Pure avoidance of unsafe moves is not optimal; timing a small concession to keep control of the long run is the right trade when it is available [1], [7]. In a constrained, well-structured domain like this, transparent heuristics can outperform deeper generic search at similar or lower compute budgets while remaining interpretable.

4 Project Accomplishment

4.1 Overview and Scope

This chapter reports *what we finished*, not how we built it (see Chapter 2 for design details). We built a reusable Dart rules engine for two players with fixed turn and scoring rules and deep cloning; a Flutter UI that runs the same on Android and on the web; online play using Firebase Realtime Database (RTDB) as an append-only move log; three simple, explainable agents (Random, Heuristic1, Deep2) that plug into the engine; and a tournament runner with analysis scripts that produce the results in Chapter 3. The same engine drives local human play, human–agent matches, online games, and headless evaluations [3]–[5].

4.2 What We Actually Built

Rules engine (Dart). The engine keeps two edge grids and one box-owner grid. It checks move legality, awards extra turns on capture, and supports *deep cloning* so agents can try moves on copies without touching the live game. If two devices apply the same sequence of moves, they end in the same state.

User interface (Flutter). The UI turns taps/clicks into engine calls and draws the board and scores. All rules live in the engine, so Android and Web builds behave the same [3], [4].

Online mode (RTDB). We rely on three built-in Firebase Realtime Database behaviours: live listeners, automatic reconnect, and offline write queueing with delivery after reconnect [5]. Because we have these guarantees, we use RTDB only to *carry* moves while keeping the engine as the source of truth. Concretely, each move is written as a single record in a room’s append-only list; both devices subscribe, set the actor from the record, and then apply the move in the engine. This simple flow avoided conflicts and kept both devices in the same state, even after brief disconnects.

Agents as plug-ins. Agents are small pure functions over engine state: *Random* picks a legal edge at random; *Heuristic1* avoids moves that let the *next* actor take a box immediately; *Deep2* looks one move ahead for both sides (two ply) and scores “our immediate gain minus the opponent’s best immediate reply.” All agents run on cloned states to keep the live game correct.

4.3 Technical Highlights (why this worked)

Two choices paid off: *same-state replay* and *clean separation*. With same-state replay, RTDB only carries moves and the engine applies them; because there is no hidden randomness, the same inputs produce the same state on every device. With clean separation, the engine has no Flutter or network dependencies, so Android and Web behave the same and headless testing is straightforward.

4.4 Research and Product Contributions

We show that *transparent heuristics with shallow lookahead* can play Dots and Boxes well on mobile-class hardware, in line with endgame advice on timing hand-outs and managing chains [1], [7]. In Chapter 3, **Deep2** beats both baselines across 4×4 – 6×6 - see the ordered win-rate

heatmaps (Figs. 6–8), where Deep2 vs Heuristic1 and Deep2 vs Random are decisive in both play orders. **Heuristic1**’s “no third sides” rule beats **Random** but shows a start effect (e.g., on 6×6 the H1–Random cells differ by order; Fig. 8). The *why* is visible in the supporting metrics: Deep2 produces longer one-turn runs (Fig. 11) while keeping risk controlled compared with payoff (Figs. 9 and 10). On the product side, online games stay consistent through short disconnects because moves are transported and replayed in the engine (Fig. 5; see also §3.8); agents swap without UI changes since the UI is engine-driven (Fig. 12).

4.5 Execution Quality

Headless matches exercised legality, scoring, extra turns, and agent hand-off; device tests showed the Android and Web builds behave the same (see Fig. 12). Online, applying moves as “set the actor from the record, then apply the move” removed the only correctness issue we saw during testing (sequence in Fig. 5).

We also logged per-move timing in the game CSVs used for Chapter 3. Medians for **Deep2** were about **0.40 ms** (4×4), **1.21 ms** (5×5), and **3.84 ms** (6×6), with 95th percentiles up to **16.34 ms** on 6×6. **Heuristic1** medians were about **0.39 ms**, **1.18 ms**, and **2.18 ms**, with 95th percentiles up to **12.09 ms**. Engine “apply” times were effectively zero-median across sizes (95th percentile ≤ 1 ms on 6×6). These timings match the claim that move selection is comfortably fast for a turn-based UI (see also §3.8).

4.6 Reflection and Implications

The results suggest that timing small concessions works better than trying to avoid them completely. The ordered heatmaps show Deep2 performing strongly in both start orders (Figs. 6–8). The unsafe and margin plots point to calculated risk paying off (Figs. 9 and 10), and the longest streak plot shows that stronger play tends to produce longer one turn runs (Fig. 11). We did not label double cross events directly, so this is a reading of the patterns rather than a claim about specific moves. Because Android and Web replay the same move records with the same engine, these patterns reflect the agents rather than platform differences (Fig. 5 and Fig. 12).

4.7 Limitations

We evaluated only 4×4–6×6 boards. Deep2 is a two-ply heuristic; we did not compare to deeper minimax/MCTS or learning agents. We used longest streak as a proxy for chain harvesting, not

explicit chain/loop detection or parity analysis. User feedback came from forms and informal playtests, not a controlled study.

5 Conclusions and Future Work

5.1 Summary of Results

5.1.1 Implementation overview

The aim was to show that a single Dart and Flutter codebase can deliver an interpretable set of AI agents for Dots and Boxes, a reproducible evaluation pipeline, and a cross-platform product experience. We implemented a modular rules engine in pure Dart that owns legality, turn rotation, scoring, and deep cloning via `clone()`, independent of the Flutter UI, which handles rendering and input [3], [4]. For online play we used Firebase Realtime Database as a minimal append-only move bus, with client-side turn enforcement and deterministic replay on both devices, so correctness follows from the engine rather than from remote arbitration [5].

5.1.2 Agents and evaluation pipeline

The engine supports three agents with increasing structure. *Random* samples uniformly from legal edges. *Heuristic1* performs a one-ply safety check and avoids moves that give the next actor an immediate box, reflecting “do not make third sides” advice from endgame practice [1], [7]. *Deep2* evaluates each legal move with two-ply lookahead: our immediate gain minus the opponent’s best immediate gain across replies, with an explicit penalty when the reply would take two or more boxes (a double-cross scenario) [1].

5.1.3 Agents and evaluation pipeline

The engine supports three agents with increasing structure. *Random* samples uniformly from legal edges. *Heuristic1* performs a one-ply safety check and avoids moves that give the next actor an immediate box, reflecting “do not make third sides” advice from endgame practice [1], [7]. *Deep2* evaluates each legal move with two-ply lookahead: our immediate gain minus the opponent’s best immediate gain across replies, with an explicit penalty when the reply would take two or more boxes (a double-cross scenario) [1]. The evaluation pipeline emits both a per-pairing CSV and a per-game

CSV. We cross-check that any summary tables can be reproduced from the per-game logs (see §3.2), and we then build heatmaps, error bars, and boxplots from those data.

5.1.4 Key findings from results

Across all board sizes the ordering is stable: **Deep2** > **Heuristic1** > **Random**. Deep2 achieves decisive win rates against both baselines in ordered pairings, including on 6×6 ; Heuristic1 beats Random with a consistent mid-50s percent P1 edge. Self-play cells cluster near 50%, with deviations attributable to first-move effects and sampling variance rather than rules errors. This pattern matches the theory. A two-ply value that prices the opponent’s immediate reply and suppresses easy double-crosses will dominate a one-ply “avoid third sides” policy, and both will outperform unstructured play [1], [7], [8].

5.1.5 Unsafe moves, streak analyses, and performance

Unsafe-move and streak analyses explain the gap in outcomes. Heuristic1 records the fewest unsafe moves, which fits its goal, but it also produces shorter streaks. Deep2 sometimes accepts a locally unsafe move when the two-ply calculation shows that the forced reply sets up a longer run, which is the sacrifice-then-harvest pattern described in expert play [1], [7]. In short, minimising unsafe moves alone does not maximise wins; timing matters. Engine-only performance is comfortably interactive. On 6×6 , median selection times are a few milliseconds (about 2.5 ms for Heuristic1 and 3.8 ms for Deep2 in our runs), with p_{95} below roughly 22 ms, which leaves ample headroom for a 60 Hz UI on typical mid-range phones since selection is not per-frame. Online synchronisation showed sub-second propagation, queued offline writes, and deterministic convergence on both devices [5]. Together, the architecture and measurements support responsive play and interpretable agent behaviour on the target platforms [3], [4].

5.2 Contributions and Implications

5.2.1 Research demonstration

The work shows that domain-informed heuristics, tailored to chain risk and timing in Dots and Boxes, can match or exceed deeper generic search within the scope studied, while remaining explainable and fast on mobile-class hardware. Deep2’s two-ply objective with an explicit double-cross penalty encodes expert intuitions without large search trees, bespoke rollout policies, or board-specific tuning [1], [7]–[9].

5.2.2 Architecture perspective

A clean separation between a pure Dart engine and a Flutter UI yields a reusable platform. New agents are pure functions over cloned state, which makes experimentation safe and reproducible. The same engine powers local human play, headless tournaments, and online matches, which simplifies product development and supports consistent research runs. The dual-CSV benchmarking pipeline reduces analysis errors and enables cross-checks between per-game distributions and aggregates, producing publication-ready figures [3], [4].

5.2.3 Practical implication

For discrete, structured games where expertise is long - chains, hand outs, double - cross and shallow - lookahead heuristics can offer a competitive edge at small compute budgets and remain interpretable [1], [7].

5.3 Future Directions

There are four natural extensions. First, *formal chain reasoning*: replace the fixed double-cross penalty with explicit chain and loop detection plus parity accounting in the sense of Berlekamp, Conway, and Guy. That would make chain handling first-class rather than emergent and should improve endgame choices and explainability [1], [2].

Second, *hybrid depth with bounds*: integrate a small, bounded search that activates only at tactical swing points identified by chain features, for example a shallow minimax or limited-budget MCTS, so we capture high-leverage moments without compromising responsiveness [8].

Third, *state-space scaling*: extend evaluation to larger boards such as 7×7 to 10×10 , using pragmatic pruning like ignoring edges far from tension, incremental deltas over the edge grids, or cached adjacency to keep selection time in check.

Fourth, *user-oriented evaluation and generalisation*: run structured human vs. AI studies to measure perceived difficulty, learning, and enjoyment, using the existing logging to analyse move timing and typical errors; then port the engine pattern—clean state model, pluggable agents, CSV-first evaluation—to other combinatorial games to probe how well these heuristics transfer [9].

5.4 Final summary

We achieved the goals of the project. One codebase delivers cross-platform play, interpretable agents, and a reproducible evaluation pipeline. The evidence and timings indicate that domain-informed heuristics with shallow lookahead are both strong and usable in practice. The path ahead is clear: make chain theory first-class inside the agent, add bounded depth where it matters, scale boards carefully, and bring human studies into the loop [1], [2], [8].

References

- [1] E. R. Berlekamp, J. H. Conway, and R. K. Guy, *The Dots-and-Boxes Game: Sophisticated Child's Play*. Natick, MA: A K Peters/CRC Press, 2001, isbn: 978-1568811291 (cited on pp. 8–14, 17, 19–21, 23, 29, 30, 32–35, 40).
- [2] D. Allcock, “Best play in dots and boxes endgames,” *International Journal of Game Theory*, vol. 50, no. 3, pp. 671–693, 2021. doi: 10.1007/s00182-020-00730-4. arXiv: 1811.10747. [Online]. Available: <https://web.ma.utexas.edu/users/allcock/research/dots.pdf> (cited on pp. 8–11, 13, 14, 34, 35).
- [3] Flutter Team. “Supported deployment platforms.” (2024), [Online]. Available: <https://docs.flutter.dev/reference/supported-platforms> (cited on pp. 8, 12–15, 19, 29, 30, 32–34, 37).
- [4] Flutter Team. “Build for and integrate with multiple platforms.” (2024), [Online]. Available: <https://docs.flutter.dev/platform-integration> (cited on pp. 8, 12–15, 19, 29, 30, 32–34, 37).
- [5] Firebase Team. “Realtime database.” (2024), [Online]. Available: <https://firebase.google.com/docs/database> (cited on pp. 8, 13–17, 19, 27, 29, 30, 32, 33, 37–40).
- [6] A. Niu, “A winning strategy for dots and boxes,” McGill University Directed Reading Program report, 2022. [Online]. Available: https://www.math.mcgill.ca/gsams/drpf/papers/papers2022/2022Winter_Niu.pdf (cited on pp. 8, 11, 12, 14).
- [7] K. Buzzard and M. Ciere, “Playing simple loony dots-and-boxes endgames optimally,” arXiv:1305.2156, 2013. doi: 10.48550/arXiv.1305.2156. arXiv: 1305.2156. [Online]. Available: <https://arxiv.org/abs/1305.2156> (cited on pp. 11–13, 17, 20, 21, 23, 29, 30, 32–34).
- [8] S. Krebbers, “Monte carlo tree search for dots-and-boxes,” M.S. thesis, Leiden University, 2021. [Online]. Available: <https://theses.liacs.nl/pdf/2020-2021-KrebbersS.pdf> (cited on pp. 12, 14, 17, 29, 33–35).

- [9] A. Pandey, "Solving dots & boxes using reinforcement learning," M.S. thesis, Colorado State University, 2022. [Online]. Available: <https://mountainscholar.org/bitstreams/539f5ef7-8035-4dda-be85-1f207195ccb4/download> (cited on pp. 12, 15, 33, 34).
- [10] Wikipedia contributors. "Dots and boxes." (2024), [Online]. Available: https://en.wikipedia.org/wiki/Dots_and_boxes.

Appendices

A Toolchain & Engine API Surface

Build toolchain (project reality)

The implementation uses **Flutter/Dart** for a single codebase targeting Android and Web; the game engine is a pure Dart package with no Flutter dependencies, which keeps rules and state transitions identical across targets [3], [4]. Online play uses **Firestore Realtime Database (RTDB)** for a minimal append-only move log with offline queuing and deterministic replay, which avoids operating a bespoke backend and keeps correctness inside the engine [5]. In development we pinned typical Android settings (e.g., `compileSdk 34`, `minSdk 23`) and normalised RTDB snapshots on Web to avoid casting issues; none of these change the engine behaviour, they simply harden builds and listeners.

Engine API surface (public, stable)

The engine exposes a compact, reproducible interface; agents and the UI both depend only on this surface. Signatures are illustrative (Dart).

```
1 class Game {
2   // Construction & cloning
3   Game(int n);           // n x n board
4   Game clone();          // deep copy for simulation
5
6   // Turn & scoring
7   int currentPlayerIndex; // actor to play
8   int score(int player);  // box count for player
9
10  // Legal moves & application
11  Iterable<Edge> availableEdges(); // unused edges
12  bool isLegalEdge(int x1, int y1, int x2, int y2);
13  bool playEdge(int x1, int y1, int x2, int y2); // applies, updates boxes, keeps turn on capture
14
15  // State layout (serialisable)
16  bool[][] hEdges; // (n+1) x n
17  bool[][] vEdges; // n x (n+1)
18  int?[][] owners; // n x n, box owners or null
19 }
20
```

```

21 // Agent contract (pure function on a clone)
22 typedef MoveFn = List<int> Function(Game g); // returns [x1,y1,x2,y2]

```

This surface is intentionally small. `clone()` enables safe lookahead; `availableEdges()` and `playEdge(...)` give agents everything they need without letting them mutate live state; and the layout supports $O(1)$ edge validation and local box checks, which keeps both heuristics and shallow lookahead real-time on 4×4 – 6×6 .

B Code and Repository

Source (engine, agents, app, evaluation): https://github.com/kanav2503/dots_and_boxes_ws.

Quick start (Flutter app).

```

git clone https://github.com/kanav2503/dots_and_boxes_ws
cd dots_and_boxes
flutter pub get
flutter run          # or: flutter run -d chrome

```

Engine tests (Dart).

```

cd packages/game_engine
dart pub get
dart test

```

Build (optional).

```

flutter build apk --release    # Android
flutter build web             # Web bundle in build/web

```

C RTDB Paths & Example Snapshot

The online transport uses a *single authoritative stream*: `rooms/<roomId>/moves` is append-only; each record contains edge endpoints and the mover identity. On receipt, the client first sets the recorded actor and then replays the edge in the engine. This keeps both devices identical across disconnects [5].

Data contract

- rooms/<roomId>/players: map of {uid: displayName}.
- rooms/<roomId>/started: boolean gate to begin play.
- rooms/<roomId>/moves/<pushId>: one move record:

```
1 {
2   "playerIndex": 0,           // or "senderId": "<uid>" in older rooms
3   "x1": 1, "y1": 2,
4   "x2": 2, "y2": 2
5 }
```

Real snapshot (truncated from the actual export)

```
1 {
2   "rooms": {
3     "-OX1YAZjajez8UYXzYKY": {
4       "players": { "6aHZ...": "malhotra", "jwGkz...": "Kanav" },
5       "started": true,
6       "moves": {
7         "-OX1YRyUsbhr4DtZg9TQ": { "playerIndex": 0, "x1": 0, "x2": 1, "y1": 1, "y2": 1 },
8         "-OX1YTEOXIA8-n_KaVBz": { "playerIndex": 0, "x1": 1, "x2": 1, "y1": 1, "y2": 2 },
9         "-OX1YVP6m_7mXtWVi-7S": { "playerIndex": 0, "x1": 1, "x2": 2, "y1": 2, "y2": 2 }
10        // ... many more moves ...
11      }
12    }
13  }
14 }
```

Clients enforce turns locally and *refuse* to write out-of-turn; the engine rejects reused edges; and RTDB ordering plus offline queues guarantee deterministic convergence when connectivity resumes [5].

D Benchmark Data Schema (for Reproducibility)

We retain two CSVs per run: a per-game file used as ground truth (visualisations, statistics) and a per-pairing summary used for quick checks. Keeping both prevents silent aggregation errors and enables richer plots.

CSV excerpts (tabular, selected columns)

Table 1. Per-game CSV excerpt (first 12 rows; selected columns).

Grid	P1 AI	P2 AI	Game #	P1 score	P2 score	P1 unsafe	P2 unsafe	P1 longest	P2 longest
4.0	Random	Random	0.0	14.0	2.0	18.0	9.0	8.0	3.0
4.0	Random	Random	1.0	9.0	7.0	15.0	14.0	5.0	4.0
4.0	Random	Random	2.0	6.0	10.0	15.0	18.0	3.0	4.0
4.0	Random	Random	3.0	11.0	5.0	14.0	12.0	4.0	4.0
4.0	Random	Random	4.0	14.0	2.0	19.0	12.0	8.0	2.0
4.0	Random	Random	5.0	11.0	5.0	17.0	16.0	4.0	4.0
4.0	Random	Random	6.0	7.0	9.0	15.0	16.0	3.0	4.0
4.0	Random	Random	7.0	12.0	4.0	19.0	14.0	10.0	3.0
4.0	Random	Random	8.0	8.0	8.0	13.0	16.0	3.0	4.0
4.0	Random	Random	9.0	8.0	8.0	15.0	16.0	5.0	4.0
... only first 12 rows shown ...									

Columns not shown: per-move timing statistics etc. Full CSV is provided in the project repository.

Table 2. Pairing summary CSV excerpt (first 12 rows; selected columns).

Grid	Agent A	Agent B	Games	A wins	B wins	Ties	$p_{A \rightarrow B}(\%)$	$p_{B \rightarrow A}(\%)$	$s(A, B)(\%)$	$\Delta(A, B)(\%)$
4.0	Random	Random	300.0	150.0	127.0	23.0	54.5	45.5	50.0	0.0
4.0	Random	Heuristic1	300.0	134.0	142.0	24.0	48.6	59.6	44.5	8.2
4.0	Random	Deep2	300.0	0.0	300.0	0.0	0.0	100.0	0.0	0.0
4.0	Heuristic1	Random	300.0	167.0	113.0	20.0	59.6	48.6	55.5	8.2
4.0	Heuristic1	Heuristic1	300.0	156.0	131.0	13.0	54.6	45.4	50.0	0.0
4.0	Heuristic1	Deep2	300.0	0.0	300.0	0.0	0.0	100.0	0.0	0.0
4.0	Deep2	Random	300.0	300.0	0.0	0.0	100.0	0.0	100.0	0.0
4.0	Deep2	Heuristic1	300.0	300.0	0.0	0.0	100.0	0.0	100.0	0.0
5.0	Random	Random	300.0	151.0	149.0	0.0	50.3	49.7	50.3	0.6
... only first 12 rows shown ...										

Percentages exclude ties. s and Δ as defined in §3.3.

E Glossary & User Feedback (Forms)

Glossary

Unsafe move: a play that allows the *next actor* to score immediately with a single reply; our code detects this by simulating one reply from the resulting state (the same check used in `givesBoxToOpponent`).

Chain / long run: a sequence of captures within a single turn, typically created by hand-outs; we proxy this via *longest scoring streak* per game on the sizes we study [1].

Double-cross: a reply that yields two or more boxes to the opponent; Deep2’s evaluation penalises enabling such rejoinders.

Order-invariant strength $s(A, B)$: $\frac{1}{2}(p_{A \rightarrow B} + (1 - p_{B \rightarrow A}))$, a single verdict that removes start advantage.

First-move bias $\Delta(A, B)$: $p_{A \rightarrow B} - (1 - p_{B \rightarrow A})$, the start advantage within the pair.

RTDB: Firebase Realtime Database; we use it as an append-only bus with offline queuing and deterministic replay [5].

User feedback (forms summary)

Form responses and informal tests report that turns feel immediate on mid-range phones and that scoring and turn feedback is clear. The most frequent requests were a shorter, simpler room-ID flow and some extra UI polish. Despite these asks, players reported high enjoyment when playing online with friends and when trying the three agents (Random, Heuristic1, Deep2). These items target product UX; the engine and Firebase Realtime Database synchronisation worked reliably throughout.

Questions used. We asked the following questions in the feedback form; the table that follows is a compact summary of the responses for the report:

1. Which platform did you use to play the game? (Android / Web / Windows / macOS / iOS)
2. Which mode did you play? (Versus AI / Online / Local two-player)
3. Which agents did you try? (Random / Heuristic1 / Deep2)
4. How responsive did the app feel overall? Please rate 1–5.
5. Was the scoring and “whose turn” feedback clear? Please rate 1–5.
6. If you used online multiplayer, how would you rate that experience? Please rate 1–5. If you did not use it, mark N/A.
7. How satisfied were you overall with the app? Please rate 1–5.
8. Did you face any difficulties or frustrations? If yes, what were they?
9. What improvements or new features would you like to see next?

Table 3. User feedback summary (n=8). Higher numbers indicate better ratings.

User	Platform	Mode	Agents tried	Responsiveness (1–5)	Score/turn clarity (1–5)	Multiplayer (1–5/NA)	Overall (1–5)	Difficulties	Suggested improvements
User 1	Android phone	Vs AI	Heuristic1, Deep2	5	5	NA	4	—	Slight UI polish; larger tap targets
User 2	Web (Chrome)	Online	Heuristic1	5	4	4	5	Room code entry took time	Shorter room-ID flow; copy button
User 3	Windows laptop	Online	Deep2	4	4	3	3	Room join a bit confusing	Clearer join screen; visual polish; stronger AI option
User 4	Android phone	Vs AI	Random, Deep2	5	5	NA	4	—	Add difficulty setting
User 5	Web (Chrome)	Online	Heuristic1	5	4	4	4	Brief reconnect message	Clearer reconnect UI; room-ID hint text
User 6	Windows desktop	Vs AI	Random	5	4	NA	4	—	Bigger tap areas; highlight last move
User 7	Android phone	Online	Deep2	5	5	5	5	—	Themes/skins for variety
User 8	Web (Firefox)	Vs AI	Random, Heuristic1, Deep2	5	5	NA	4	—	Short tutorial on chains